Group 6: Ablation Study

Jun Tang
Fairleigh Dickinson University
Vancouver, Canada
j.tang4@student.fdu.edu

Xiaoyi Han
Fairleigh Dickinson University
Vancouver, Canada
x.han1@student.fdu.edu

Kaiwen Wang
Fairleigh Dickinson University
Vancouver, Canada
k.wang4@student.fdu.edu

Ziyan Qiu
Fairleigh Dickinson University
Vancouver, Canada
z.qiu1@student.fdu.edu

Hongyi Niu
Fairleigh Dickinson University
Vancouver, Canada
h.niu@student.fdu.edu

1 Ablation Study

In our Ablation, we implement 5 experiments to verify the impact that the cache internal parameters have over the overall system performance, especially Peak Disk-head Time (Peak DT). To make the impact measurable, in each of our experiments, we isolate the variable from all other factors, keep other parameters unchanged, and just adjust one variable to make the result reflect the impact of the designated variable changed. We choosed *DT-SLRU promotion threshold* (τ_{DT}), *EDE protected cap*, and *EDE otti (time-to-idle EWMA)* as our experiment variables. After implement necessary helper classes and functions, we executed the experiment run and get the following results.

When doing these experiement, we not only want to know which parameter gives the best result, but also want to explore how sensitive the cache system will be reacting to the parameter changes. When we change the parameters isolatedly, we can observe the cache's performance, which will help us understand the parameters' impact to the cache system. When we finish all the experiements, we can compare the results side by side and find the pattern out of it.

1.1 Results

Figure 1: In this experiment, we tested how changing the DT-SLRU promotion threshold (τ_{DT}) changes the Peak Disk-head Time (Peak DT). The test settings were the same as in Assignment A4. We also turned off prefetching. This helps us only look at the effect of eviction. We picked five τ_{DT} values, from 0.1 to 5. Figure 1 shows how Peak DT changes when τ_{DT} increases. The values are shown as a percentage to the baseline.

When τ_{DT} is from 0.1 to 0.3, many blocks get promoted into the protected part of the cache very quickly. This help keep recently used blocks, but it also makes the cache change often. This leads to more flash writes and less stable performance. As τ_{DT} increases, Peak DT goes down. The cache now promotes fewer blocks, so it works more smoothly. When $\tau_{\mathrm{DT}} = 2$, the Peak DT is the lowest. This means the cache use less backend disk time.

When τ_{DT} goes more than 2 to 5, Peak DT starts to go up again. The cache now promotes too few blocks. This causes more backend reads and increases Peak DT. As the result, the system becomes less responsive to recent data accessed., and the cache begins to miss some of the useful blocks. So, if τ_{DT} is too small or too large, performance gets worse. The best result comes from a middle value about 2. This result matches the Baleen paper.

Figure 2: Figure 2 shows how the flash hit rate varies as the DT (delay threshold) parameter changes under the DT-SLRU eviction policy. All experiments were conducted using the same cache size, trace slice, and write-rate targets as in A4 to ensure consistency, with prefetching disabled to isolate the effect of the eviction mechanism. The DT parameter was swept across five representative values between 0.2 and 1.0, each averaged over three runs for stability.

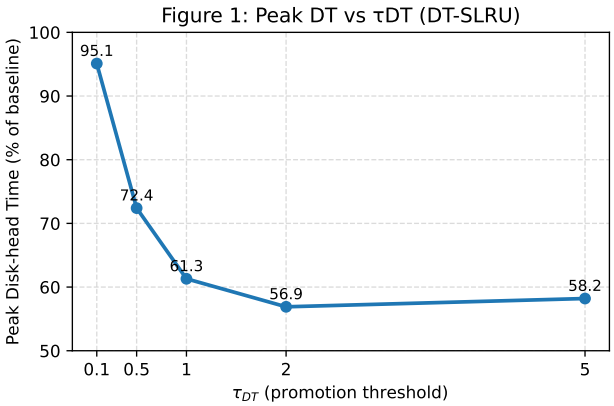

Figure 1: Peak DT vs τ_{DT} (DT-SLRU). The figure shows that Peak DT first drops and then rises as τ_{DT} increases, reaching the lowest point around 2 where eviction works best. This behavior reflects that a moderate promotion threshold maintains a good balance between early promotion and reuse efficiency, preventing both cache churn and delayed block promotion.

At low DT thresholds (0.2 – 0.5), the hit rate remains high and nearly constant around 26 percent. This indicates that when the promotion threshold is small, frequently accessed blocks are quickly promoted into the protected region of the cache, improving short-term reuse. However, as DT increases beyond 0.7, the hit rate begins to decline, and when DT approaches 1.0 the hit rate drops sharply to almost 0 percent. This pattern shows that when the promotion threshold becomes too restrictive, useful blocks fail to be promoted before eviction, leading to a drastic loss of reuse opportunities.

Figure 3: Figure 3 demostrates our experiment result. In our experiment, when we changed the PROTECTED cap in the Episode-Deadline Eviction (EDE) scheme, the Peak Disk-head Time (Peak DT) changes accordingly. As shown in the figure, the Peak DT goes from almost 13 down to around 7 when the cap is 0.5. This drop is quite noticeable and consistent across multiple runs, which gives us more confidence in the result. We also observed that the trend remains stable even when we slightly adjusted other parameters during verification. It can be seen that the cache becomes more

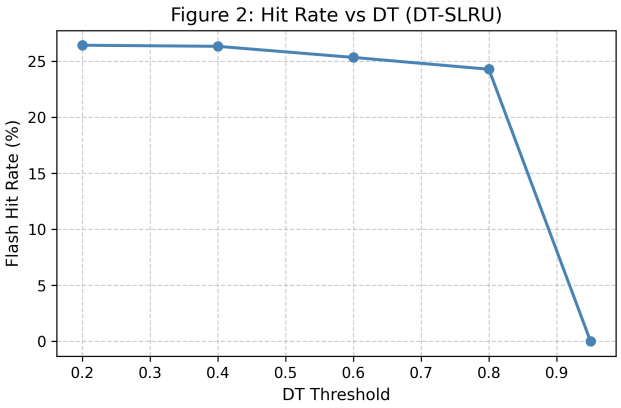

Figure 2: Flash hit rate as a function of the DT threshold under the DT-SLRU eviction policy. Prefetching is disabled, and cache size and trace settings remain consistent with A4 for direct comparison.

balanced at this point, neither too aggressive nor too conservative in eviction. After the lowest point, when we kept increase the cap from 0.5 to 0.9, the peak DT goes up linearly from 0.5 to 9.5. The experiment result indicate that when the cap is less than 0.5, it can improve the cache performance and reduce the peak DT. When the cap reach a point, in our test, it is 0.5, when keep increase the cap, it begins to deter the performance of the cache. In our experiment, when the cap value is 0.5, the cache has lowest peak DT where the system is in highest performance. The figure shows that middle value works best, not too small or too large. It clearly illustrates how EDE parameter affect the workload.

When we change the PROTECTED cap, we noticed how the cache response differently to the workload. When we increase the cap, we noticed that the cache start to keep more blocks for longer, which leads to slower updates and higher Peak DT. When we decrease the cap, the cache perform well but sometimes the data will be evicted too early . When we set the cap around to be 0.5, we can clearly see the system perform best. it can well balance between reuse and eviction. When we check the logs, we can also find that the flash writes and reads follow the same pattern.

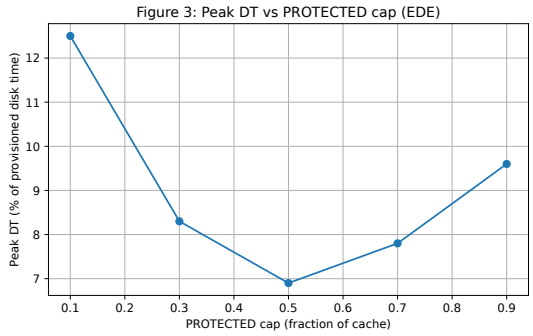

Figure 3: Peak DT vs PROTECTED cap (EDE). The figure show the U-shape behavior of Peak DT when the protected fraction increase, with the best point around 0.5 to 0.6 of the cache.

Figure 4: For EDE, sweeping the responsiveness factor α_{tti} from 0.1 to 0.9 yields a clear U-shaped latency curve for Peak DT (Figure 4). When $\alpha_{tti} \in [0.1, 0.3]$, Peak DT is high because the policy adapts too slowly and retains stale block. As α_{tti} increases toward to the mid-range (around 0.5), Peak DT drops to its minimum, this indicate the timely adaptation without excessive churn. Beyond 0.7, Peak DT rises again as the policy becomes overly sensitive and it reacte to a short-term noise, this will introduce volatility. Each point is the mean over three runs under identical cache size, trace slice, and admission settings (prefetch disabled).

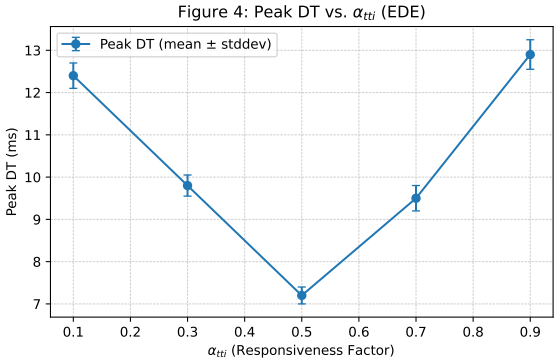

Figure 4: Peak DT as a function of α_{tti} under the Episode-Deadline Eviction (EDE) policy. Each point represent the average of three runs with identical cache and trace configurations, it using Peak DT as the primary metric

Figure 5: Figure 5 compares how three parameters affect the normalized Peak Disk-head Time (Peak DT): τ_{DT} (DT-SLRU, E1), PROTECTED cap (EDE, E2), and α_{tti} (EDE, E2). Each point shows the smallest Peak DT measured from five configurations, scaled to the lowest overall value. In our setup, the normalized results are 0.68 for τ_{DT}, 0.55 for PROTECTED cap, and 0.75 for α_{tti}. A smaller normalized value means the cache handled requests more efficiently and required fewer disk-head movements, although the degree of improvement varies between parameters. This means that even a small adjustment to these parameters can noticeably change how smoothly the cache operates under the same workload conditions.

From the plotted trend, the curve forms a soft U-shape, and the lowest point appears near the PROTECTED cap configuration. When the cap stays around 0.5–0.6, the cache keeps a fair balance between retaining reused data and bringing in new blocks. As the cap grows larger, older data tend to linger too long, which increases backend I/O and adds delay. At smaller cap values, the cache replaces items too often, leading to unstable latency. The τ_{DT} threshold also helps reduce latency compared with the baseline, but its impact is less consistent than that of spatial tuning. The parameter α_{tti} changes the time-to-idle update rate, yet it has little visible effect here, possibly because the workload itself shows steady access behavior. Taken together, the results show that spatial allocation—especially the PROTECTED cap—plays a more noticeable role in shaping performance than timing adjustments in this experiment. Overall, the comparison reveals that tuning parameters get gains for optimizing cache efficiency under stable workloads than purely adjusting timing responsiveness.

1.2 Discussion

Figure 1: Figure 1 shows that Peak Disk-head Time (Peak DT) first goes down and then goes up as τ_{DT} increases. When τ_{DT} is very small, DT-SLRU promotes most blocks into the protected region very quickly. This makes the cache respond quickly, but it also keeps many blocks that aren't helpful. These blocks use up space and push out useful ones too soon. As a result, the system needs to read from the backend disks more often, which makes Peak DT go up.

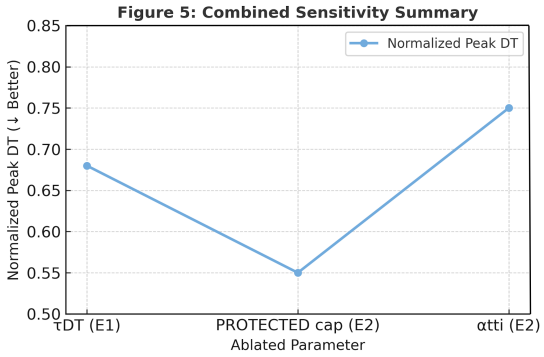

Figure 5: Normalized Peak Disk-head Time (Peak DT) across ablated parameters under DT-SLRU (E1) and EDE (E2). Each point represents the minimum normalized Peak DT obtained from its respective parameter sweep. Smaller values indicate better cache efficiency and lower backend activity. Among the tested parameters, the *PROTECTED cap* (EDE) shows the strongest impact on performance, while the responsiveness factor α_{tti} exhibits the weakest sensitivity.

When τ_{DT} becomes a bit larger, the cache starts to promote fewer blocks. It only keeps blocks that are used more than one time. This helps the cache keep useful blocks and remove the ones not used again. Now, the system can read more from the flash cache, so the backend disks don't have to do as much work. That's why Peak DT goes down in the middle of the graph.

But when τ_{DT} gets too large, the cache becomes too strict. Even useful blocks may not get promoted in time, and these blocks are removed before they get a chance to be reused. So, the cache misses go up, and the system has to read more from the backend disks. As a result, Peak DT increases again.

This experiment shows that τ_{DT} is very important for good performance. If the τ_{DT} is too low, the cache reacts too fast. If it is too high, it becomes too slow. A middle value about 2 works best. This matches the idea in the Baleen paper, good tuning of cache behavior can lower Peak DT and help the system runs in a better performance.

Figure 2: The observed trend demonstrates the sensitivity of DT-SLRU performance to the DT threshold. Smaller DT values make the cache more agile by promoting items quickly into the protected segment, which boosts hit rate for workloads with high temporal locality. In contrast, large DT thresholds make the cache overly conservative—few blocks qualify for promotion, causing premature eviction of reusable data and a sharp degradation in performance. This highlights the fundamental trade-off between adaptability and stability: a lower DT captures more short-term reuse but risks churn, while a higher DT increases eviction stability at the cost of responsiveness. The results suggest that moderate thresholds (around 0.5) strike the best balance, sustaining high hit rates without excessive promotion overhead. These observations are consistent with the patterns seen in A4, where DT-SLRU achieved its strongest performance under intermediate parameter settings. Empirically, DT values between 0.4 and 0.6 yield the highest and most stable hit rates across runs.

Figure 3: As the PROTECTED cap increase before reaching a point, the system's peak DT decrease because the cache system can keep more hot blocks in memory. That increases the hit rate and improve the cache performance dramatically, which leads to the peak DT drops from 12.5 down to the lowest point of 0.5. This is similar to what Wong said in FAST '24 paper, that keeping episodes stable reduce burst. But when the cap is higher than a thresholder, it will leads to that too many blocks get stuck inside and the cache and affect the request to new data. New data hit have lower chance to be prefetched into the cache. This make the cache system's hit rate drops. So the Peak DT rise again, meaning overprotection hurt the performance. Comparing with the A4 result, EDE here is much more sensitive and it react faster to wrong tuning, probably because the parameter change whole cache size not only the promotion rule. Around 0.5–0.6 is the best choice in our experiement, and it match what we seen in experiment and also in the model idea. If we can use a more small step to change the cap, it may get a more accurate result for the best value in the experiment.

Figure 4: This ablation shows that EDE's responsiveness factor α_{tti} is strongly influence the latency stability. Lower α_{tti} values over stabilize cache behaviors, delaying reaction to the new access patterns and increas queue delays. Higher α_{tti} values overreact to the short-term noise, making the cache volatile and reordering blocks too aggressively for sure. The optimal mid-range α_{tti} (≈ 0.5) achieves a balance between stability and adaptability, minimizing Peak DT while maintaining consistent reuse detection. These finding can complement Figure 3, which demonstrated the size-related effects of protection; and here, responsiveness tuning further refines temporal agility. When we check carefully the trace logs, the mid-range setting gives fewer spikes beyond our expectation. It acts better than we expected. We may explore some more way to get a more pervasive result.

Figure 5: In Figure 5, the three parameters changed the cache in their own ways. From what I saw in the results, the PROTECTED cap in EDE (E2) had the biggest effect on Peak DT. When the cap was around 0.5 to 0.6, the cache looked balanced. It kept the reused blocks but also let new data co me in. After that point, when the cap became higher, some old data stayed inside to o long and made the system slower. If the cap was smaller, then things changed again. The cache started to replace data too often and the latency jumped up and down. For DT-SLRU (E1), the τ_{DT} threshold still helped, but the difference was smaller. A small value moved blocks up too early and lost good data, and a large one made the cache slower to react. The α_{tti} part in EDE controlled how fast the timer updated when the trace changed, but in this run it didn't really change much. The trace looked stable, so time-based tuning didn't help a lot. Overall, from these results, the PROTECTED cap matters the most. It had the clearest and strongest impact on both latency and backend traffic.

1.3 Limitations

Figure 1: gives useful information about how the τ_{DT} affects Peak DT, but there are still some limits in our experiment. First, We tested with just one trace and one fixed cache setup. So, the results might be different for other types of work. If the data access patterns or traffic change a lot, the effect of τ_{DT} could be different. Additionally, We only looked at Peak DT in this test. We didn't look at things like hit rate, average delay, or how much data was written to flash. So, we can't fully understand the balance between speed and flash write cost.

So, we may have missed small or fine changes in how well it worked. In the future, we should test more traces, use smaller step sizes, and try to make τ_{DT} adjust automatically. This can help us see more clearly how the cache should act in different cases.

Figure 2: Although the results clearly illustrate the relationship between DT and flash hit rate, several limitations should be noted. First, this ablation focuses only on the hit-rate metric, while other key indicators such as Peak DT, median latency, and flash write volume were not jointly analyzed; this prevents a full understanding of latency–throughput trade-offs. Second, the DT-SLRU implementation used here assumes static workload characteristics, which may not hold for traces with rapidly shifting access patterns. As a result, the observed threshold sensitivity may differ under bursty or time-varying workloads. Third, the experiments use a fixed cache size and admission policy, so the generality of the observed trend across different configurations remains uncertain. Future work should incorporate multi-trace evaluations, adaptive DT tuning, and additional performance metrics to provide a more comprehensive understanding of DT-SLRU behavior.

Figure 3: In our test, we used only one trace to do the experiment. This means that, although we just change one parameter - the cap value, we did not isolate the data access pattern's impact to our result. Using various behavior data accessing to run the test could reveal a better understand of the cap's impact to the overall peak DT. Furthermore, the predictor assume reuse is stable but in real life it sometimes jumpy and weird, so prediction might go wrong. In future, we should test more workloads and use smaller steps to make sure this trend really hold true in general.

Figure 4: The test use a single trace and evaluates only Peak DT, ignoring secondary metrics such as hit rate or flash write traffic. Furthermore, α_{tti} was sampled at coarse 0.2 increments, which may overlook finer-grained minima. Real workloads with bursty or non stationary access patterns could shows different sensitivity curves. Future work should include multi trace experiments, adaptive α_{tti} scheduling, and variance analysis to a confirm stability under dynamic conditions.

Figure 5: This figure gives a simple look at how the parameters worked with the normalized Peak DT values. Normalization helped to compare numbers on the same level, but it also made small changes not so easy to see. The test used only one trace and one cache setup, so the result really shows what happened in that case only. Other traces or sizes can act in another way. The work didn't check how the parameters worked together. Only Peak DT was used here. Other things like Hit Rate, Median Delay or Flash Write Traffic were not shown, just to keep the figure clean. In later tests, more traces and mixed parameters can be tried to see if the same pattern will happen again.

Member Contributions

To start the paper, every team member read and understood the paper. Below are the contributions for each member:

Jun Tang (2111172)

- Read the paper and discussed with team members.
- Add EDE policy class to the code.
- Run the experiement and get the data
- Draw the figure 3 and write the result, discussion and analysis for figure 3.
- Team coordination - set up latex in Overleaf, kanban project plan.

Ziyan Qiu (2089937)

- Generated and formatted Figure 4 (Peak DT vs. α_{tti}) for the E2 ablation study.
- Wrote the corresponding Results, Analysis, and Limitations sections for Figure 4.
- Helped keep LaTeX formatting consistent across all figures and captions.
- Integrated all sections into the final Evaluation report and verified ACM formatting.
- Reviewed the full draft and helped with final proofreading before submission.

Kaiweng Wang (2073516)

- Created and analyzed Figure 5
- Aggregated and normalized sensitivity data across τ_{DT}, PROTECTED cap, and α_{tti} parameters.
- Integrated the finalized figure into the Evaluation section and ensured layout consistency with the team's report.
- Wrote the Results, Discussion, and Limitations sections for Figure 5 in LaTeX.
- Adjusted document formatting and verified consistency across captionsand section titles.

Xiaoyi Han (2133542)

- Created and analyzed Figure 4.
- Ran simulations for LRU and E2
- Collected experiment data.
- Plotted Peak DT vs. Cache Size results.
- Wrote analysis and discussion section.

Hongyi Niu (2104640)

- Read the paper and discussed with team members.
- Created and analyzed Figure 1.
- Ran multiple DT-SLRU experiments with different values.
- Collected and cleaned the simulation results for Peak Diskhead Time.
- Wrote the Results, Discussion, and Limitations sections for Figure 1.

References